

SECURITY REVIEW

FOR PULSE IN PRIVATE



INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	SCOPE	04
03	AUDITORS	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	M-01 Rescue Tokens: Implement a Rescue Funds Function for stucked funds	09
	M-02 Constant Fee for Relayer	10
08	DISCLAIMER	11
09	ABOUT KANN AUDITS	12

01 - OVERVIEW

AUDIT SUMMARY

Pulse In Private engaged Kann Audits to perform a security review of the Pulse In Private smart contracts. A team of 2 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

2

 ISSUES

CRITICAL	-
HIGH	-
MEDIUM	02
LOW	-
INFORMATIONAL	-

ENGAGEMENT DETAILS

PREPARED FOR	Pulse In Private
PREPARED BY	Kann Audits
DATE REPORTED	28 March 2025
METHODS	Manual review

02 - WHAT WE REVIEWED

SCOPE

REPOSITORY	https://github.com/alexbabits/pip
AUDITED COMMIT	ce32639

#	FILE
01	/src/Pip.sol
02	/circuits/height12/withdraw.circom

03 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

KA

Kann

LEAD SECURITY RESEARCHER

IV

Ivan Fitro

LEAD SECURITY RESEARCHER

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL -	● HIGH -	● MEDIUM 02	● LOW -	● INFO -
TOTAL 02 FIXED 01 ACKNOWLEDGED 01				

ID	TITLE	SEVERITY	STATUS
M-01	Rescue Tokens: Implement a Rescue Funds Function for stucked funds	● MEDIUM	● ACKNOWLEDGED
M-02	Constant Fee for Relayer	● MEDIUM	● FIXED

DETAILED FINDINGS

M-01

Rescue Tokens: Implement a Rescue Funds Function for stucked funds

SEVERITY

● MEDIUM

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

In Pip.sol, when a user deposits, they receive a nullifier code which is required when withdrawing from a different address. However, since it is possible for a user to forget their nullifier code, the funds could become permanently stuck in the contract.

■ RECOMMENDATION

Add a rescueFunds function along with an on-chain view function to check whether the nullifier hash (used during withdrawal) has been consumed. This would enable the protocol to verify that the user genuinely forgot their nullifier code and facilitate the recovery of their funds.

■ REMEDIATION

● ACKNOWLEDGED

M-02

Constant Fee for Relayer

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

Gas consumption on the chain could increase - either temporarily during high congestion periods or permanently due to evolving network conditions. With the current fixed fee parameters, relayers may not receive sufficient incentives to cover their gas expenses. In some cases, a relayer might end up paying more in gas fees than the reward they receive. This imbalance not only discourages relayer participation but also forces new addresses to source gas from elsewhere in order to call the `withdraw()` function, which negatively impacts the protocol's overall usability and efficiency.

■ RECOMMENDATION

User-Defined Relayer Fee: Allow users to input a custom relayer fee when initiating a transaction. This approach gives users the flexibility to determine what they consider a fair incentive for the relayer, based on current network conditions and their own willingness to pay.

Dynamic Gas Fee Function: Alternatively, implement a function that can adjust the gas fee for relayers. This function would allow the protocol administrator (or even potentially the community through governance mechanisms) to update the relayer fee dynamically in response to changes in gas prices. This ensures that the fee remains competitive and sufficient to cover relayer costs.

■ REMEDIATION

● FIXED

08 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits